# 02-4、接口自动化Agent开发实战(RAG双存储)

## 1、格式化接口数据

在前面的章节中我们设计好了api文档处理之后的数据格式如下，那么不管是何种形式的接口文档，最新我们都需要进行格式化处理成我们期望的格式。

```
{
    "project":"所属项目名称",
    "module":"所属模块名称",
    "tag":"接口的标签",

    "method":"请求方法",
    "path":"接口路径",
    "summary":"接口名称",
    "description":"接口写详细的业务描述",
    "parameters":"查询参数说明",
    "requestBody":"请求体的说明",,
    "responses":"响应体的说明",

    "interface_id":"接口的唯一ID，存储向量数据库的uuid",

    "updated_at":"更新时间",
    "version":"版本号"
}
```

### 1、Curl清洗提示词模版

```
# @Author  : 木森
# @weixin: python771
from langchain.prompts import PromptTemplate


# curl接口数据清洗
curl_cleansing_prompt = PromptTemplate(
    input_variables=["input_text"],
    template="""
    请你从以下抓包的 CURL 命令中提取 API 接口信息为结构化数据，按照以下 JSON 模板输出接口信息：

    提取规则要求如下
        1. 如无相关信息可留空或使用 null；
        2. 如果没有提供 response 示例或描述，则 responses 字段请输出为空数组：[];
        3. 优先提取真实字段名与示例值，尽可能还原接口结构；

    输出格式模板如下：
        {{
          "method": "HTTP方法(GET/POST/PUT/DELETE等)",
```

```
          "path": "API路径",
          "summary": "接口的简要名称",
          "description": "详细的业务描述，合并description和operationId等信息",
          "parameters": {{
            "query": [{{
              "name": "参数名",
              "type": "参数类型",
              "required": "是否必填",
              "description": "参数说明",
              "example": "示例值"
            }}],
            "path": [{{
              "name": "参数名",
              "type": "参数类型",
              "required": "是否必填",
              "description": "参数说明",
              "example": "示例值"
            }}],
            "header": [{{
              "name": "字段名称",
              "type": "参数类型",
              "required": "是否必填",
              "description": "参数说明",
              "example": "示例值"
            }}]
          }},
          "requestBody": {{
            "content_type": "请求体类型（如application/json）",
            "schema": [{{
                "name": "字段名",
                "type": "字段类型",
                "required": "true/false",
                "description": "字段说明+字段约束描述"
              }}
            ],,
            "example": "示例值",
            "required": "是否必填",
            "description": "请求体说明"
          }},
          "responses": [
            {{
              "status_code": "200",
              "description": "成功响应描述",
              "schema": "数据结构描述",
              "example": "示例值"
            }},
            {{
              "status_code": "400",
              "description": "错误响应描述",
              "schema": "错误数据结构",
              "example": "错误示例"
            }}
          ]
        }}
    示例输入：
        curl -X GET "http://api.example.com/user?id=123" -H "Accept:
application/json"
    示例输出：
```

```
        {{
          "method": "GET",
          "path": "/user",
          "summary": "获取用户信息",
          "description": "通过用户ID获取用户的详细信息",
          "parameters": {{
            "query": [{{
              "name": "id",
              "type": "string",
              "required": "true",
              "description": "用户唯一标识",
              "example": "123"
            }}],
            "path": [],
            "header": []
          }},
          "requestBody": {{
            "content_type": null,
            "schema": null,
            "example": null,
            "description": null
          }},
          "responses": [
            {{
              "status_code": "200",
              "description": "返回用户信息",
              "schema": "User 对象",
              "example": "{{\\"id\\": \\"123\\", \\"name\\": \\"Alice\\"}}"
            }},
            {{
              "status_code": "400",
              "description": "参数错误",
              "schema": "Error 对象",
              "example": "{{\\"error\\": \\"invalid id\\"}}"
            }}
          ]
        }}
    请处理下面这个接口:
        {input_text}
    """
)
```

## 2、OpenAPI接口数据清洗

```python
# @Author  : 木森
# @weixin: python771

from langchain.prompts import PromptTemplate

swagger_cleaning_prompt = PromptTemplate(
    input_variables=["input_text"],
    template="""
    你是一个接口分析专家，请你从下方提供的 OpenAPI / Swagger 接口结构（已是 JSON 格式）中提取接口关键信息，
    并转换为统一结构化 JSON 格式
```

```
    ### 规则说明
        - 请求体中每个字段都需标注 required，若文档无明确说明，默认ture。
        - 其它字段及规则同之前描述。
        - 无请求体时，requestBody 各字段均填 null。
        - 响应字段无须 required，只需 type 和 description。

    ### 输出格式要求如下：
        {{
          "method": "HTTP方法(GET/POST/PUT/DELETE等)",
          "path": "API路径",
          "summary": "接口的简要名称",
          "description": "详细的业务描述，合并 description 和 operationId 等信息",
          "parameters": {{
            "query": [],
            "path": [],
            "header": []
          }},
          "requestBody": {{
            "content_type": "请求体类型，如 application/json",
            "schema": [{{
                "name": "字段名",
                "type": "字段类型",
                "required": "true/false",
                "description": "字段说明+字段约束描述"
              }}
            ],
            "example": "请求示例",
            "description": "请求体说明"
          }},
          "responses": [
            {{
              "status_code": "状态码",
              "description": "响应描述",
              "schema": "响应结构说明",
              "example": "示例内容"
            }}
          ]
        }}
    请处理以下接口数据，输出结构化结果
    {input_text}
""")

if __name__ == '__main__':
    from agents.models import deepseek_v3_llm

    chain = swagger_cleaning_prompt | deepseek_v3_llm
    result = chain.invoke({"input_text": """
     {
        "path": "/api/cron/crontab/{id}",
        "method": "PUT",
        "summary": "修改定时任务的配置",
        "description": "",
        "parameters": [
            {
                "name": "id",
                "in": "path",
                "required": true,
                "type": "string",
```

```
                "description": ""
            }
        ],
        "requestBody": {
            "name": {
                "type": "string",
                "description": "任务名称"
            },
            "state": {
                "type": "boolean",
                "description": "是否启用"
            },
            "run_type": {
                "type": "string",
                "description": "任务类型"
            },
            "interval": {
                "type": "integer",
                "description": "执行间隔时间"
            },
            "date": {
                "type": "string",
                "description": "指定执行的事件"
            },
            "crontab": {
                "type": null,
                "description": "周期性任务规则"
            }
        },
        "responses": {
            "200": {
                "type": null,
                "description": ""
            },
            "422": {
                "detail": {
                    "type": "array",
                    "description": ""
                }
            }
        },
        "tag": [
            "定时任务"
        ]
    }

    """})
    print(result.content)
```

## 3、文本数据接口文档清洗模版

```
# @Author  : 木森
# @weixin: python771

from langchain.prompts import PromptTemplate
```

```
swagger_cleaning_prompt = PromptTemplate(
    input_variables=["input_text"],
    template="""
    你是一个接口分析专家，请你从下方提供的接口文本描述中提取接口关键信息，
    并转换为统一结构化 JSON 格式

    ### 规则说明
        - 请求体中每个字段都需标注 required，若文档无明确说明，默认ture。
        - 其它字段及规则同之前描述。
        - 无请求体时，requestBody 各字段均填 null。
        - 响应字段无须 required，只需 type 和 description。

    ### 输出格式要求如下：
        {{
          "method": "HTTP方法(GET/POST/PUT/DELETE等)",
          "path": "API路径",
          "summary": "接口的简要名称",
          "description": "详细的业务描述，合并 description 和 operationId 等信息",
          "parameters": {{
            "query": [],
            "path": [],
            "header": []
          }},
          "requestBody": {{
            "content_type": "请求体类型，如 application/json",
            "schema": [{{
                "name": "字段名",
                "type": "字段类型",
                "required": "true/false",
                "description": "字段说明+字段约束描述"
              }}
            ],
            "example": "请求示例",
            "description": "请求体说明"
          }},
          "responses": [
            {{
              "status_code": "状态码",
              "description": "响应描述",
              "schema": "响应结构说明",
              "example": "示例内容"
            }}
          ]
        }}
    请处理以下接口数据，输出结构化结果
    {input_text}
""")

if __name__ == '__main__':
    from agents.models import deepseek_v3_llm

    chain = swagger_cleaning_prompt | deepseek_v3_llm
    result = chain.invoke({"input_text": """
        ### 1.4 修改用户信息
        ```
        PUT /api/users/{userId}
        ```
        **请求参数**:
```

```
        | 参数名    | 类型    | 必填 | 描述                        | 示例
                |
        | -------- | ------ | ---- | ------------------------- | ----------------
------------------- |
        | nickname | string | 否   | ≤20字符                     | "新昵称"
                |
        | avatar   | string | 否   | 图片URL                     |
"https://mstest.com/new-avatar.jpg" |
        | phone    | string | 否   | 手机号                      | 13800138000
                |
        | gender   | string | 否   | "MALE"/"FEMALE"/"SECRET" | "MALE"
                |
        **响应示例**:
        ```json
        {
          "message": "更新成功"
        }
        ```
    """})
    print(result.content)
```

## 4、数据清洗工具封装

```
# @Author  : 木森
# @weixin: python771
import json
import random
import time
import uuid
from langchain_core.output_parsers import JsonOutputParser
from agents.models import deepseek_v3_llm
from rag.prompt.curl_clearnsing_prompt import curl_cleansing_prompt
from rag.prompt.swagger_clearnsing_prompt import swagger_cleaning_prompt
from rag.prompt.text_cleansing_prompt import text_cleaning_prompt
from pydantic import BaseModel


# 提取清洗之后的json格式数据
class ApiTerFaceJsonData(BaseModel):
    """解析大模型清洗后的json数据格式"""
    method: str
    path: str
    summary: str
    description: str
    parameters: dict
    requestBody: dict
    responses: list


class ApiDataCleansing:
    """接口文档数据清洗的类"""

    @staticmethod
    def get_interface_uuid(method: str, path: str) -> uuid.UUID:
```

```
        """
        根据接口的请求方法和路径+毫秒级时间戳生成一个uuid值
        :param method:
        :param path:
        :return:
        """
        # 获取毫秒级的时间戳
        t = str(int(time.time() * 1000))
        text = t + method + path
        return uuid.uuid3(uuid.NAMESPACE_DNS, text)

    @staticmethod
    def api_data_cleansing(data: str | dict, document_type: str) -> dict:
        """
        对接口文档进行数据清洗，整理为统一的数据格式
        :param data:
        :param document_type:
        :return:
        """
        # 判断数据的类型，如果是字符串则进行json解析
        if isinstance(data, dict):
            data = json.dumps(data, ensure_ascii=False)
        # 判断接口文档的类型
        if document_type == "openapi" or document_type == "swagger":
            prompt = swagger_cleaning_prompt
        elif document_type == "curl":
            prompt = curl_cleansing_prompt
        else:
            prompt = text_cleaning_prompt

        parser = JsonOutputParser(pydantic_schema=ApiTerFaceJsonData)
        # 组装llm的调用链
        chain = prompt | deepseek_v3_llm | parser
        # 使用大模型对数据进行清洗
        result = chain.invoke({"input_text": data})
        return result


if __name__ == '__main__':
    # res = ApiDataCleansing.get_interface_uuid("POST", "/api/v1/order/submit")
    # print(res)
    data = """
       ### 1.4 修改用户信息
        ```
        PUT /api/users/{userId}
        ```
        **请求参数**:
        | 参数名    | 类型    | 必填 | 描述                        | 示例
               |
        | -------- | ------ | ---- | ------------------------- | ----------------
------------------- |
        | nickname | string | 否    | ≤20字符                     | "新昵称"
                |
        | avatar   | string | 否    | 图片URL                     |
"https://mstest.com/new-avatar.jpg" |
        | phone    | string | 否    | 手机号                      | 13800138000
                |
```

```
        | gender    | string | 否    | "MALE"/"FEMALE"/"SECRET" | "MALE"
                    |
        **响应示例**:
        ```json
        {
          "message": "更新成功"
        }
        ```
    """
    res = ApiDataCleansing.api_data_cleansing(data, "text")
    print(json.dumps(res, ensure_ascii=False, indent=4))
```

# 2、RAG数据双存储实现

## 1、新增单个接口

接口文档数据------->格式化处理（数据清洗）---->存入mysql----->存入向量数据库

```
    @staticmethod
     def add_api_document(class_name: str,
                          data: dict | str,
                          document_type: str,
                          project=None,
                          tag=None,
                          module=None,
                          version=None):
        """
         新增单个接口,
        :param class_name:向量数据库的集合名称
        :param data: 原始接口文档描述信息
        :param document_type: 文档的类型
        :param project: 项目名称
        :param tag:  接口的标签
        :param module: 接口所属模块
        :param version: 版本信息
        :return:
        """
        # 对接口文档进行数据清洗,获取接口的基本信息
        api = ApiDataCleansing.api_data_cleansing(data, document_type)
        # 生成接口的uuid
        api['interface_id'] = ApiDataCleansing.get_interface_uuid(api['method'],
api['path'])
        api['project'] = project
        api['tag'] = tag
        api['module'] = module
        api['version'] = version
        # 通过上下文协议来实现数据回滚还是提交，保证双存储数据库的一致性
        with SQLDB() as sql_db:
            # 1、存储数据到mysql中
            sql_db.add_api(api)
            # 2、向量存储
            with VectorDB(class_name) as vector_db:
```

```
                text = api['summary'] + api['method'] + api['path'] +
api['description']
                # 向量数据库新增数据
                vector_db.add_text(api['interface_id'], text, metadata=api)
```

## 2、更新接口

输入接口信息------->格式化处理（数据清洗）----更新mysql接口数据----->更新向量数据库接口数据

```
    @staticmethod
    def update_api_document(class_name: str,
                            data: dict | str,
                            document_type: str,
                            project=None,
                            tag=None,
                            module=None,
                            version=None
                            ):
        """更新单个接口"""
        # 获取接口的id数据
        uid = data['interface_id']
        # 对更新后的文档内容进行清洗
        api = ApiDataCleansing.api_data_cleansing(data, document_type)
        api['project'] = project | ""
        api['tag'] = tag | ""
        api['module'] = module | ""
        api['version'] = version | ""
        # 开启事物
        with SQLDB() as sql_db:
            # 修改mysql数据库中的数据库
            sql_db.update_api(api)
            # 修改向量数据库中的数据
            with VectorDB(class_name) as vector_db:
                text = api['summary'] + api['method'] + api['path'] +
api['description']
                # 向量数据库修改数据
                vector_db.update_text(uid, text, metadata=api)
```

## 3、删除接口

根据接口id---->删除mysql数据库中的接口信息----->删除向量数据库中接口信息

```
    @staticmethod
    def delete_api_document(class_name: str,
                            interface_id: str):
        """删除单个接口"""
        with SQLDB() as sql_db:
            # 删除mysql数据库中的数据
            sql_db.delete_api(interface_id)
            # 删除向量数据库中的数据
            with VectorDB(class_name) as vector_db:
                vector_db.delete_text(interface_id)
```

## 4、批量导入：

**解析接口数据-----遍历接口------->格式化处理（数据清洗）---->存入mysql----->存入向量数据库**

```
    def import_openapi_document(self, class_name: str, json_file_path=None):
        """openapi文档批量导入"""
        # 解析openapi的json文件
        parser = OpenAPIParser(json_file_path).parser()
        # 遍历所有的接口信息，一个一个接口进行数据清洗和存储
        for index, api in enumerate(parser):
            print(f"正在导入第{index + 1}个接口:", api['summary'])
            self.add_api_document(class_name=class_name, data=api,
document_type='openapi', project='演示项目001')
            print(f"第{index + 1}个接口导入成功")
```